Out of the box, the default services container only supports constructor injection. However, ASP.NET Core adds method parameter injection in a couple of places. You have already seen an example of this in chapter 2, when we looked at creating conventional middleware. If you recall, you injected an `ILogger<T>` into the `InvokeAsync` method:

```
public async Task InvokeAsync(HttpContext context,
➥ ILogger<IpAddressMiddleware> logger)
```

But what about handler methods? After all, handler method parameters are seen as binding targets by the model binder. What happens when the model binder encounters an `IPriceService` parameter? Your application breaks. That's what happens, unless you prefix the service parameter with the `FromServices` attribute:

```
public async Task OnGetAsync([FromServices]IPriceService service)
{
    // do something with service
}
```

This is a useful pattern for services that are expensive to create but are only used a fraction of the time in a Razor page. It might be, for example, that you have a named handler in a page that needs a service that is not required by the `OnGet` and `OnPost` handlers, and the named handler is only called under certain circumstances. In this scenario, it makes little sense to inject the service into the `PageModel` constructor. The `FromServices` attribute lets you scope the service purely to the handler method that needs it, and it will only be resolved when it is needed.

7.3.3 Directly from the service container with GetService and GetRequiredService